

Sapienza

Università di Roma

Ingegneria dell'Informazione, Informatica e
Statistica

Laurea Triennale

Dipartimento di Informatica

Sistemi Operativi

Appunti a cura di
Lorenzo Ferraro

26 settembre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it>

Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Sistemi Operativi*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale github.com/TonnoArcobaleno/uni-notes, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

Segnalazioni

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@gmail.com**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

Indice

1	Introduzione	1
1.1	Sistemi numerici	1
1.1.1	Convertire i numeri tra i diversi sistemi numerici	1
1.1.2	Numeri col segno	2
1.2	Operazioni elementari	4
1.2.1	Addizioni e sottrazioni	4
1.2.2	Il problema dell'overflow	5
1.2.3	Moltiplicazioni	5
1.2.4	Shift di bit	6

Capitolo 1

Introduzione

1.1 Sistemi numerici

Il sistema numerico usato normalmente è il sistema numerico **decimale** (o in base 10). In questo, ogni cifra rappresenta una potenza di 10: $5374_{10} = 5 \cdot 10^3 + 3 \cdot 10^2 + 7 \cdot 10^1 + 4 \cdot 10^0$. Esistono, oltre a questo sistema, anche molti altri sistemi in basi diverse, tra cui il sistema **binario** e il sistema **esadecimale**.

Definizione 1.1 – Sistema binario

Il sistema **binario** rappresenta ogni numero tramite solo due cifre: **0** e **1**. Ogni cifra rappresenta una potenza di 2: $1101_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 1 = 13$.

Definizione 1.2 – Sistema esadecimale

Il sistema **esadecimale** rappresenta ogni numero tramite tutte le **10 cifre** e tramite le lettere che vanno dalla **A** alla **F** (che rappresentano i numeri da 10 a 15). Ogni cifra rappresenta una potenza di 16: $4AF_{16} = 15 \cdot 16^0 + 10 \cdot 16^1 + 4 \cdot 16^2 = 15 + 160 + 1024 = 1199$.

1.1.1 Convertire i numeri tra i diversi sistemi numerici

Abbiamo visto come convertire qualsiasi numero in binario/esadecimale in decimale, ma vediamo come fare il contrario.

- **Decimale** $\rightarrow$ **Binario**: vi sono due metodi:

- 1° Metodo: prendere la più grande potenza di 2 più piccola del numero da voler convertire, e sottrarla, e continuare così finché non si arriva a 0. Ad esempio, $68 = 64 - 4$, dunque $68_{10} = 1000100_2$.
- 2° Metodo: dividere il numero per 2 e posizionare il resto appena ottenuto a sinistra della rappresentazione binaria. Ad esempio, $68/2 = 34 + 0 \rightarrow 34/2 = 17 + 0 \rightarrow 17/2 = 8 + 1 \rightarrow 8/2 = 4 + 0 \rightarrow 4/2 = 2 + 0 \rightarrow 2/2 = 1 + 0 \rightarrow 1/2 = 0 + 1$, dunque $68_{10} = 1000100_2$.

- **Decimale** $\rightarrow$ **Esadecimale**: dividere il numero per 16 e posizionare il resto appena ottenuto a sinistra della rappresentazione esadecimale. Ad esempio, $68/16 = 4 + 4 \rightarrow 4/16 = 0 + 4$, dunque $68_{10} = 44_{16}$.

Proprietà 1.1 – Potenze di 2

Conoscendo solo alcune potenze di 2, è possibile approssimare i valori di molte più potenze. Infatti, se si conoscono tutte le potenze da 2^0 a 2^9 , e sapendo che $2^{10} \approx 1000$, $2^{20} \approx 1000000$, $2^{30} \approx 1000000000$, ..., si possono approssimare tutte le potenze tra 2^0 e 2^{39} . Infatti, $2^{34} = 2^{30} \cdot 2^4 \approx 1000000000 \cdot 16 = 16000000000$.

Definizione 1.3 – Definizioni sistema binario

Nel sistema binario, chiamiamo **bit** le cifre dei numeri. Un **nibble**, poi, sono 4 bit mentre un **byte** sono 8 bit. Esistono poi il **most significant bit** e **least significant bit**, che rispettivamente il bit più a sinistra e più a destra, e il **most significant byte** e **least significant byte**, che sono rispettivamente il byte più a sinistra e più a destra.

Ad esempio, nel numero 1010101010, il primo 1 è il MSB, mentre l'ultimo 0 è il LSB. Allo stesso modo, nel numero CEB48E8 (rappresentato in esadecimale, dove ogni cifra rappresenta un nibble), il byte CE è il MSB, mentre il byte E8 è il LSB.

1.1.2 Numeri col segno

In qualsiasi sistema numerico, per rappresentare un numero negativo basta mettere il simbolo $-$ prima del numero. Questo, però, per i computer non è possibile: lavorando solo con dei bit, non si può anteporre ad un numero il simbolo $-$. Infatti, il computer non ha idea di cosa sia quel simbolo, conosce solo 0 e 1. Dunque bisogna trovare un'altra soluzione per rappresentare i numeri col segno. Vi sono due modi possibili:

- **Segno e Ampiezza**
- **Completamento a 2**

Proprietà 1.2 – Segno e Ampiezza

Segno e Ampiezza è una tecnica usata dai computer per rappresentare i numeri col segno. Si usa il **bit più significativo per rappresentare il segno**: se questo vale **0**, allora il numero è **positivo**, altrimenti è **negativo**. La formula per convertire un numero in decimale è la seguente:

$$A = (-1)^{a_{N-1}} \sum_{i=0}^{N-2} a_i 2^i$$

Ad esempio, il numero 6 tramite il Segno e Ampiezza diventa in binario 0110, mentre -6 diventa 1110. È dunque necessario utilizzare un bit in più del normale per rappresentare i numeri. Con N bit si possono quindi rappresentare tutti i numeri da $-(2^{N-1} - 1)$ a $2^{N-1} - 1$.

Il problema più grande di questa tecnica è l'addizione: semplicemente, questa non funziona. Se infatti proviamo a sommare -6 (1110) e 6 (0110), troviamo che

$$\begin{array}{rcccccc} 1 & 1 & 1 & 0 & + & \\ 0 & 1 & 1 & 0 & = & \\ \hline 1 & 0 & 1 & 0 & 0 & = 19_{10} \neq 0 \end{array}$$

Inoltre, un'altro problema è che vi sono due rappresentazioni dello 0: infatti, $0 = 1000 = 0000$.

Per questi motivi nasce un'altra tecnica, quella del **complemento a 2**.

Proprietà 1.3 – Complemento a 2

Complemento a 2 è un'altra tecnica usata per rappresentare i numeri col segno. Il **bit più significativo** ha lo stesso valore visto con i numeri senza segno, ma con il **segno negativo**. La formula per convertire un numero in decimale è la seguente:

$$A = a_{N-1}(-2^{N-1}) + \sum_{i=0}^{N-2} a_i 2^i$$

Ad esempio, il numero 7 tramite complemento a 2 diventa in binario 0111, mentre -7 diventa 1001. Con N bit si possono quindi rappresentare tutti i numeri da -2^{N-1} a $2^{N-1} - 1$.

Con questa implementazione si risolvono entrambi i problemi del Segno e Ampiezza. Infatti, lo zero è rappresentato solo dalla sequenza 0000, e l'addizione non ha problemi: infatti, se

proviamo a sommare -6 (1010) e 6 (0110), troviamo che

$$\begin{array}{r} 1 \ 0 \ 1 \ 0 \ + \\ 0 \ 1 \ 1 \ 0 \ = \\ \hline 1 \ 0 \ 0 \ 0 \ 0 \ = \ 0 \end{array}$$

dove l'uno iniziale non si conta dato che stiamo parlando di numeri a 4 bit.

Proprietà 1.4 – Convertire un numero in un numero negativo tramite il Complemento a 2

Preso un numero in binario possiamo facilmente trasformarlo nell'opposto tramite il Complemento a 2. Infatti, basta **invertire i bit** e **aggiungere 1**. Preso ad esempio $6_{10} = 0110_2$, troviamo che $-6_{10} = 1001_2 + 1 = 1010 = -6_{10}$.

Questa proprietà funziona per un motivo molto semplice: sappiamo che $6 + \bar{6} = 0110 + 1001 = 1111$, dunque aggiungendo un 1 troviamo che $6 + \bar{6} + 1 = 1111 + 1 = 10000$, e dato che il primo zero si elude, troviamo che $6 + \bar{6} + 1 = 0$, ossia $-6 = \bar{6} + 1 \rightarrow -1001 = 0110 + 1$.

1.2 Operazioni elementari

1.2.1 Addizioni e sottrazioni

Le addizioni in binario e in esadecimale funzionano esattamente uguale a come funzionano in decimale. Ad esempio,

$$\begin{array}{r} 1 \\ 0 \ 1 \ 1 \ + \\ 0 \ 1 \ 0 \ = \\ \hline 1 \ 0 \ 1 \end{array}$$

$$\begin{array}{r} 1 \\ A \ 4 \ F \ + \\ 2 \ A \ 1 \ = \\ \hline C \ F \ 0 \end{array}$$

Anche per la sottrazione il metodo rimane lo stesso. Ad esempio,

$$\begin{array}{r} 1(2) \\ 0 \ 10 \ 0 \ - \\ 0 \ 0 \ 1 \ = \\ \hline 0 \ 0 \ 1 \end{array}$$

$$\begin{array}{r} 1(16) \\ 3 \ \cancel{D} \ 9 \ - \\ 1 \ A \ D \ = \\ \hline 2 \ 2 \ C \end{array}$$

1.2.2 Il problema dell'overflow

Osservazione 1.1 – Overflow

Un problema classico nelle addizioni tra numeri binari in un sistema digitale è l'**overflow**: i computer lavorano su un numero fisso di bit, e con un addizione è possibile sfiorare questo numero.

Ad esempio, se lavoriamo con numeri a 4 bit, la somma $1001 + 1000 = 10001$ sfiora, creando diversi problemi. Infatti, con N bit abbiamo un range di numeri rappresentabili, da 0 a $2^N - 1$ (questo se parliamo di numeri senza segno).

Vi sono due modi per risolvere questo problema:

- **Sign-Extension**
- **Zero-Extension**

Proprietà 1.5 – Sign-Extension

La **sign-extension** è un'operazione usata dai computer per aumentare il numero di bit di un numero (in Complemento a 2) così da poter completare somme su di esso che superino il limite di bit prestabilito. La regola è semplice: **uplicare il bit più significativo a sinistra raddoppiando il numero di bit del numero** (raddoppiamo il numero di bit così da esser coperti sia sulle addizioni ma anche sulle moltiplicazioni, che vedremo tra poco).

Ad esempio, $3_{10} = 0011 = 00000011$, e $-5_{10} = 1011 = 1111011$. Questo con gli 0 ovviamente funziona, ma anche con gli 1: infatti, $-5 = 1011 = -8+2+1 = -16+8+2+1 = -32+16+8+2+1 = -64+32+16+8+1 = 1111011$.

Proprietà 1.6 – Zero-Extension

La **zero-extension** è un'operazione usata dai computer per aumentare il numero di bit di un numero così da poter completare somme su di esso che superino il limite di bit prestabilito. La regola è semplice: **raddoppiare la lunghezza del numero aggiungendo tutti 0 a sinistra**.

Ad esempio, $3_{10} = 0011 = 00000011$ (come nella Sign-Extension). La Zero Extension però **non funziona con i numeri col segno**. Infatti, $-5_{10} = 1011 \neq 00001011 = 11_{10}$.

1.2.3 Moltiplicazioni

Anche le moltiplicazioni in binario e in esadecimale funzionano come in decimale. Ad esempio,

$$\begin{array}{r}
 0 \ 1 \ 1 \ \times \\
 0 \ 0 \ 1 \ = \\
 \hline
 0 \ 1 \ 1 \ + \\
 0 \ 0 \ 0 \ + \\
 0 \ 0 \ 0 \ + \\
 \hline
 0 \ 0 \ 0 \ 1 \ 1
 \end{array}$$

$$\begin{array}{r}
 3 \ B \ 7 \ - \\
 2 \ C \ 4 \ = \\
 \hline
 E \ D \ C \ + \\
 2 \ C \ 9 \ + \\
 7 \ 6 \ E \ + \\
 \hline
 A \ 4 \ 6 \ 1 \ C
 \end{array}$$

1.2.4 Shift di bit

Definizione 1.4 – Shift

Uno **shift** è un'operazione sui bit che **sposta tutte le posizioni dei bit** di una sequenza binaria o a destra o a sinistra di un determinato numero di posizioni.

Esistono due tipi di shift:

- **shift aritmetico**, dove si ripete il bit più significativo seguendo la logica della sign-extension;
- **shift logico**, dove si inseriscono degli 0 a sinistra della sequenza seguendo la logica della zero-extension.

Ad esempio, $-7_{10} = 1001 \gg 2 = 0010 = 2_{10}$ è uno **shift logico** (si è infatti perso il segno, cosa che capita quando si usa la zero-extension), mentre $-8_{10} = 111000 \ggg 3 = 111111 = -1_{10}$ è uno **shift aritmetico**. Notiamo che, in entrambi i casi, il nuovo numero si può ottenere prendendo l'originale e dividendolo per 2^N , dove N è il valore dello shift. Questa è una proprietà degli shift di destra. Allo stesso modo, per gli shift di sinistra, basta moltiplicare il valore per 2^N . Ad esempio, $-8_{10} = 111000 \ll 2 = 100000 = -32 = -8 \cdot 2^2$.

Proposizione 1.1 – Calcolare numero decimale dopo uno shift

Preso un numero A_{10} , effettuando uno shift a **sinistra** di N posizioni, troviamo che $A_{10} \ll N = A_{10} \cdot 2^N$. Se invece si effettua uno shift a **destra**, troviamo che $A \gg N = A/2^N$.